

Reviewer Pack — Minimal Index of Quotient Sheaves and Resolution Proof Width...

Entry 0a09fe8bf099 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 06:38:50 UTC

Minimal Index of Quotient Sheaves and Resolution Proof Width Correlation

Entry ID: 0a09fe8bf099 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 06:38:50 UTC

1. Conjecture

Field A (mathematical branch): Sheaf Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

The minimal index of the quotient sheaves associated with a Tseitin formula φ_G is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $\min_index(Q\varphi_G) = \Theta(w(\varphi_G))$ for a given d -regular graph G .

Rationale (proposer's reasoning):

Sheaf theory provides a framework to study local properties of geometric spaces, which could potentially reveal local complexities in resolution proofs. By analyzing the quotient sheaves, we may uncover hidden structures that contribute to proof complexity.

Taxonomy category: Sheaf_Theoretic (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bec23340be30b130

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the 30 random seeds, the correlation coefficient between $\min_index(Q\varphi_G)$ and $w(\varphi_G)$ is greater than or equal to 0.7, with no seed showing a correlation coefficient less than 0.3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal index of quotient sheaves" AND "resolution proof complexity"
- "Sheaf Theory" AND "Tseitin formula resolution width correlation" - "quotient sheaves
Tseitin formula" AND "resolution proof width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(d, n):
        if d * (n - 1) % 2 != 0:
            return None
        graph = [[] for _ in range(n)]
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if len(graph[i]) < d and len(graph[j]) < d:
                    if (i, j) not in edges and (j, i) not in edges:
                        graph[i].append(j)
                        graph[j].append(i)
                        edges.add((i, j))
        return graph

    def tseitin_formula(graph):
        n = len(graph)
        literals = [f'x{i}' for i in range(n)]
        clauses = []
        for i in range(n):
            clause = [literals[i]]
            for j in graph[i]:
                clause.append(f'¬{literals[j]}')
            clauses.append(clause)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if (i, j) not in graph and (j, k) not in graph and (k, i) not in graph:
                        clauses.append([f'¬{literals[i]}', f'¬{literals[j]}', literals[k]])
```

```

    return clauses

def resolution_width(clauses):
    queue = clauses[:]
    resolvents = set()
    width = 0
    while queue:
        clause = queue.pop(0)
        if len(clause) > width:
            width = len(clause)
        for other in queue:
            common_literals = [lit for lit in clause if lit.startswith('-') and -int(lit[1:])]
            if common_literals:
                new_clause = list(set(clause + other) - set(common_literals))
                if any(new_clause[i].startswith('-') and -int(new_clause[i][1:]) == new_clause[0] for i in range(1, len(new_clause))):
                    continue
                resolvents.add(tuple(sorted(new_clause)))
            queue.append(list(resolvents.pop()))
    return width

def quotient_sheaves(graph):
    n = len(graph)
    sheaves = [set() for _ in range(n)]
    for i in range(n):
        for j in graph[i]:
            if not sheaves[i].intersection(sheaves[j]):
                sheaves[i] |= {j}
    return sheaves

def min_index(sheaves):
    n = len(sheaves)
    indices = [0] * n
    for i in range(n):
        for j in sheaves[i]:
            if not indices[j]:
                indices[j] = 1
            else:
                indices[j] += 1
    return max(indices)

n_max = 40
instances_tested = 0
total_metric_value = 0.0

for n in range(5, n_max + 1):
    graph = generate_d_regular_graph(3, n) # Example with d=3
    if not graph:
        continue

    clauses = tseitin_formula(graph)
    width = resolution_width(clauses)

    sheaves = quotient_sheaves(graph)
    min_index_value = min_index(sheaves)

    instances_tested += 1

```

```

        total_metric_value += min_index_value * width

    if instances_tested < 30:
        return {
            "metric_name": "min_index_times_width",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    mean_metric = total_metric_value / instances_tested
    std_metric = (sum((x - mean_metric) ** 2 for x in range(5, n_max + 1)) / instances_tested) ** 0.5

    return {
        "metric_name": "min_index_times_width",
        "metric_value": mean_metric,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric = (sum((r["metric_value"] - mean_metric) ** 2 for r in results if r["metric_value"] is not None) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(r["counterexample"] == "not_enough_instances" for r in results):
        print("RESULT: INCONCLUSIVE not_enough_instances")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"not_enough_instances\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd152430.py", line 135, in <module>

```

result = run_trial(seed)
^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd152430.py", line 99, in run_trial
    width = resolution_width(clauses)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd152430.py", line 60, in resolution_wi
    common_literals = [lit for lit in clause if lit.startswith('-') and -
int(lit[1:]) in other]
                                                                    ^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'x1'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution before producing data that could be analyzed to determine if the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code, then rerun the test with a larger sample size to ensure robustness of the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	35786
2	preregistration	ollama_remote	glm4:latest	0	0	9683
3	novelty	ollama_remote	glm4:latest	0	0	8302
4	novelty	ollama_remote	glm4:latest	0	0	8929
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20358
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25797
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10987
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21122
9	judge	ollama_remote	glm4:latest	0	0	9167

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 150132 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0a09fe8bf099.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/0a09fe8bf099.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0a09fe8bf099.tar.gz` (if generated)